CS 423
Operating System Design:
Persistence:
Storage Devices and IO Scheduling
Oct 29

Ram Alagappan

LOGISTICS

MP3 due by Nov 11, 11:59 CT

Midterm grades released, this week is the deadline for regrades

Any questions about midterm?

Final exam dates and location (see Piazza)

AGENDA / LEARNING OUTCOMES

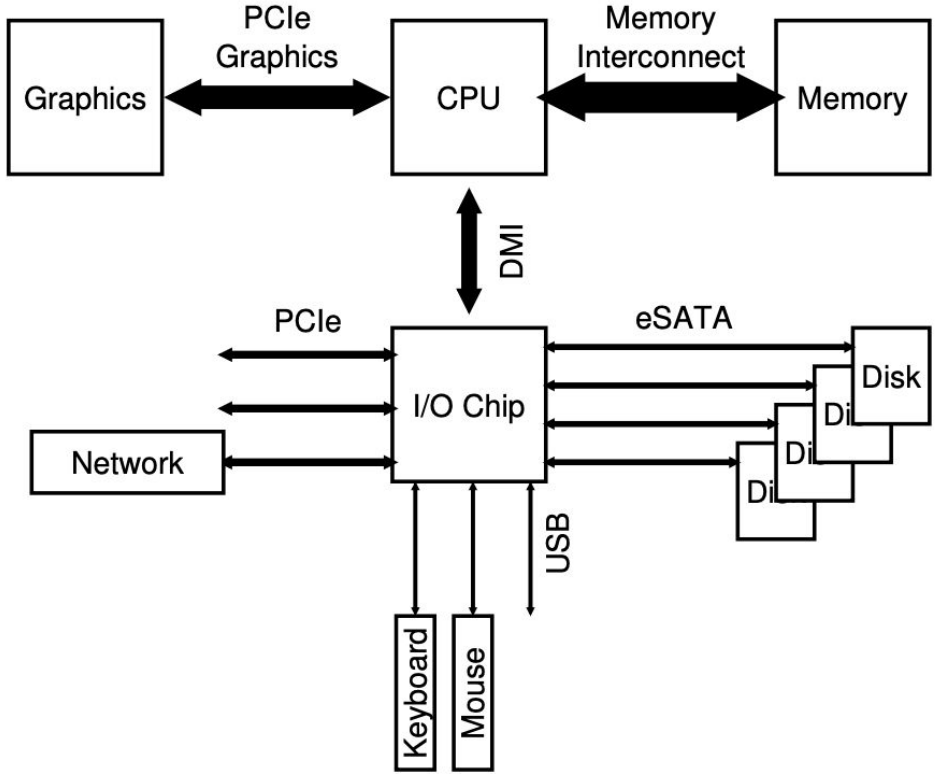
Basics of persistence

General IO devices

Storage device: HDDs

IO Scheduling

ARCHITECTURE DIAGRAM



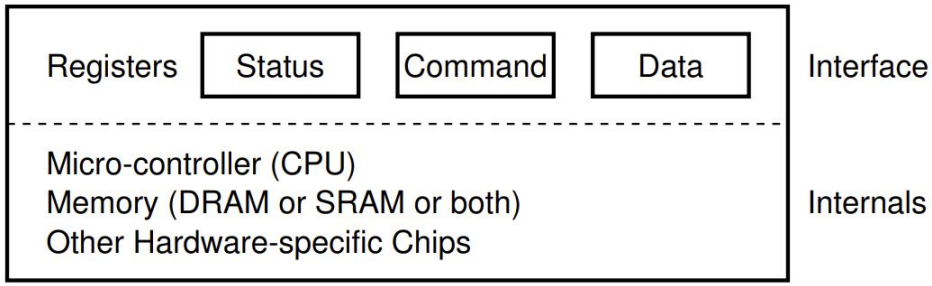
DMI: direct media i/f

- dedicated graphics
- PCIe
  - network, monitor
  - NVMe SSD
- USB
  - kb, mouse, etc
- eSATA/SCSI
  - HDD, SSD

CPU PCIe lanes = fastest/direct path
Chip PCIe lanes = indirect/shared path through the chipset

DEVICE INTERFACE AND INTERNALS

A simplified model




```
While (STATUS == BUSY)
    ; // wait until device is not busy
Write data to DATA register
Write command to COMMAND register
    (starts the device and executes the command)
While (STATUS == BUSY)
    ; // wait until device is done with your request
```

POLLING VS. INTERRUPT

Previous example - CPU polling
What's the problem with polling?

Interrupts to the rescue

When does polling make sense?

An hybrid approach – poll + interrupt
(does this look familiar?)

PIO VS. DMA

CPU directly transferring data – programmed IO

Wasteful because CPU is some trivial transfers instead of running useful programs

Solution: Direct Memory Access (DMA)

OS tells the *DMA engine* about the transfer and CPU goes on to do useful work - inform about which memory, where data in memory, how many bytes to copy, etc.

When DMA is done, it interrupts the CPU

In modern PCIe devices, the DMA capability is often integrated into the device controller rather than being a separate chip

General principle: separation of concerns - control plane vs. data plane.
CPU setting up the transfer - control
DMA directly transferring bytes in/out of memory - data

DEVICE DRIVERS

Problem:
- many, many devices
- each has its own protocol

How can we avoid writing a slightly different OS for each H/W combination?

Write device driver for each device

Drivers are 70% of Linux source code (and most OS bugs reside here!)

DEVICE DRIVERS

Each OS defines *class interface*
• Network, block, sound

Driver exports class interface to OS

Driver uses device-specific interface to talk to device
– Storage: USB storage, SATA, NVMe
– Keyboard/mouse

Must be written separately for each Linux, Windows, MacOS

STORAGE DEVICE

Components

- storage space: flash or magnetic media
- memory: buffer to stage data to/from device
- controller: CPU + registers (status/cmd/etc)

Data is transferred in units of blocks

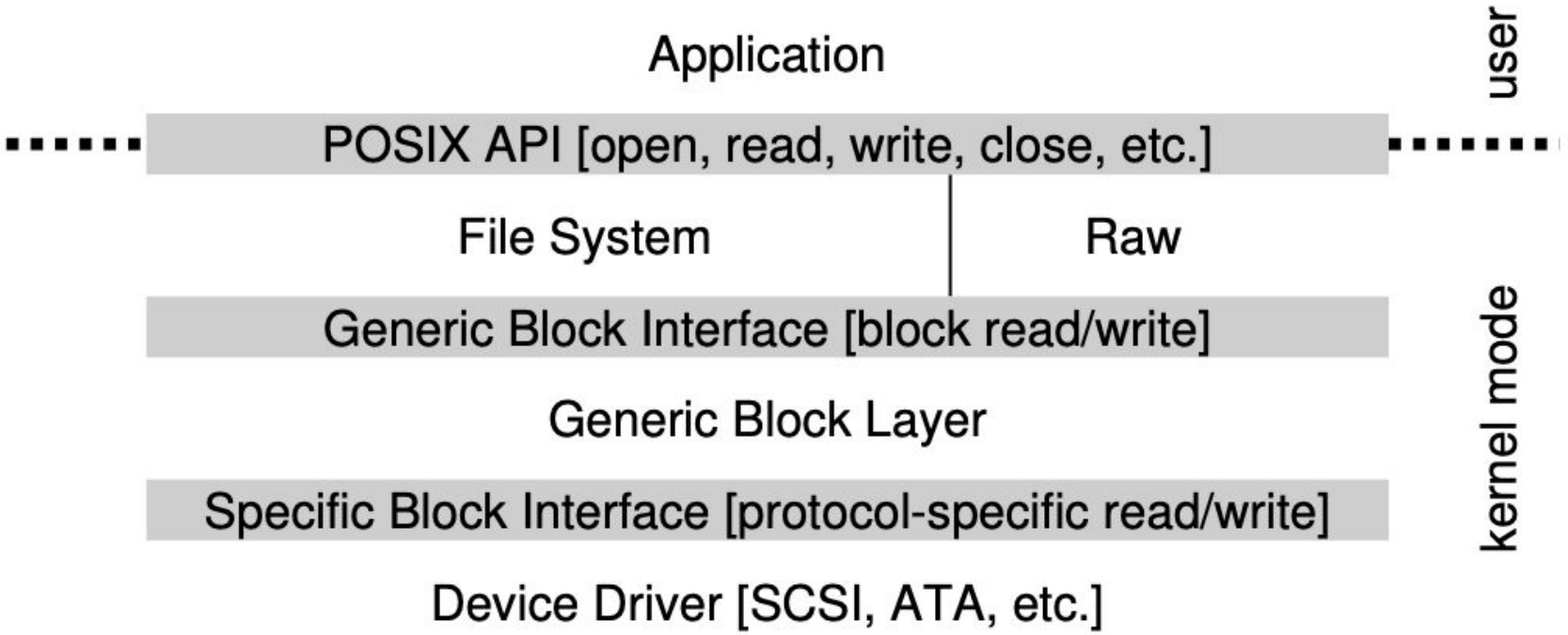
- Logical Block Address (LBA)

OS communication with device

- Programmed I/O vs DMA
- Completion: interrupt vs polling

Device Drivers

FILE SYSTEM STACK

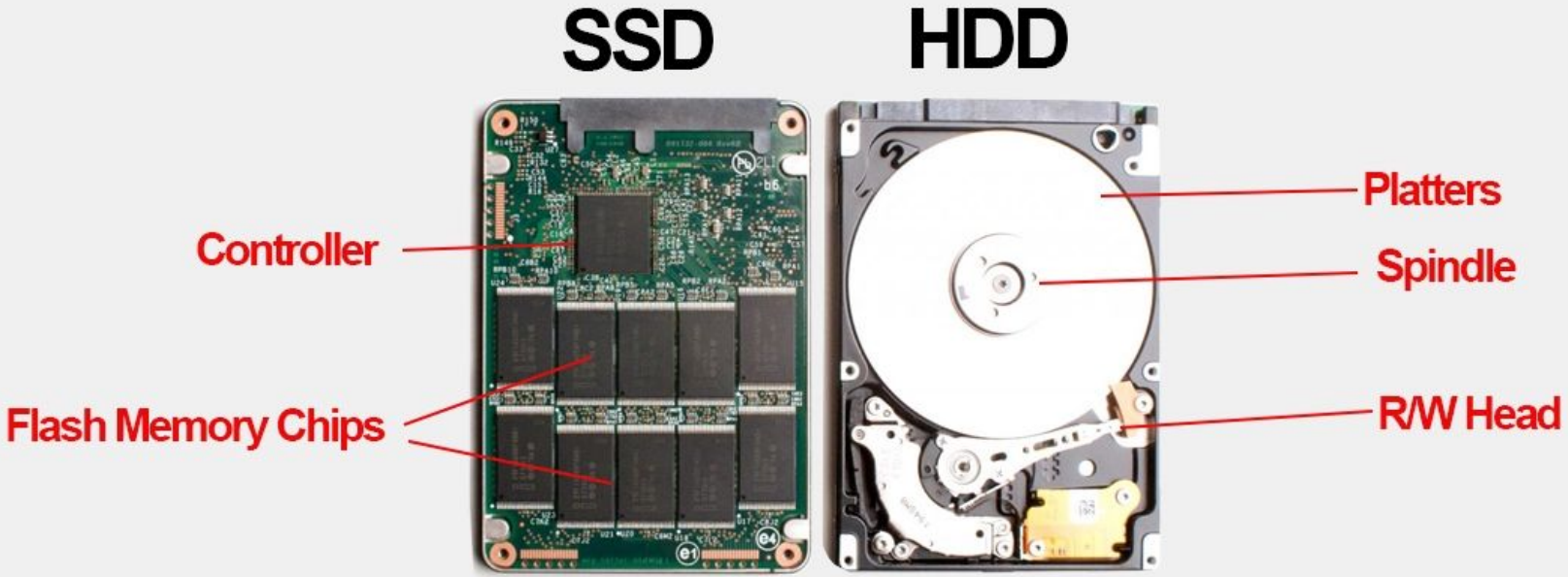


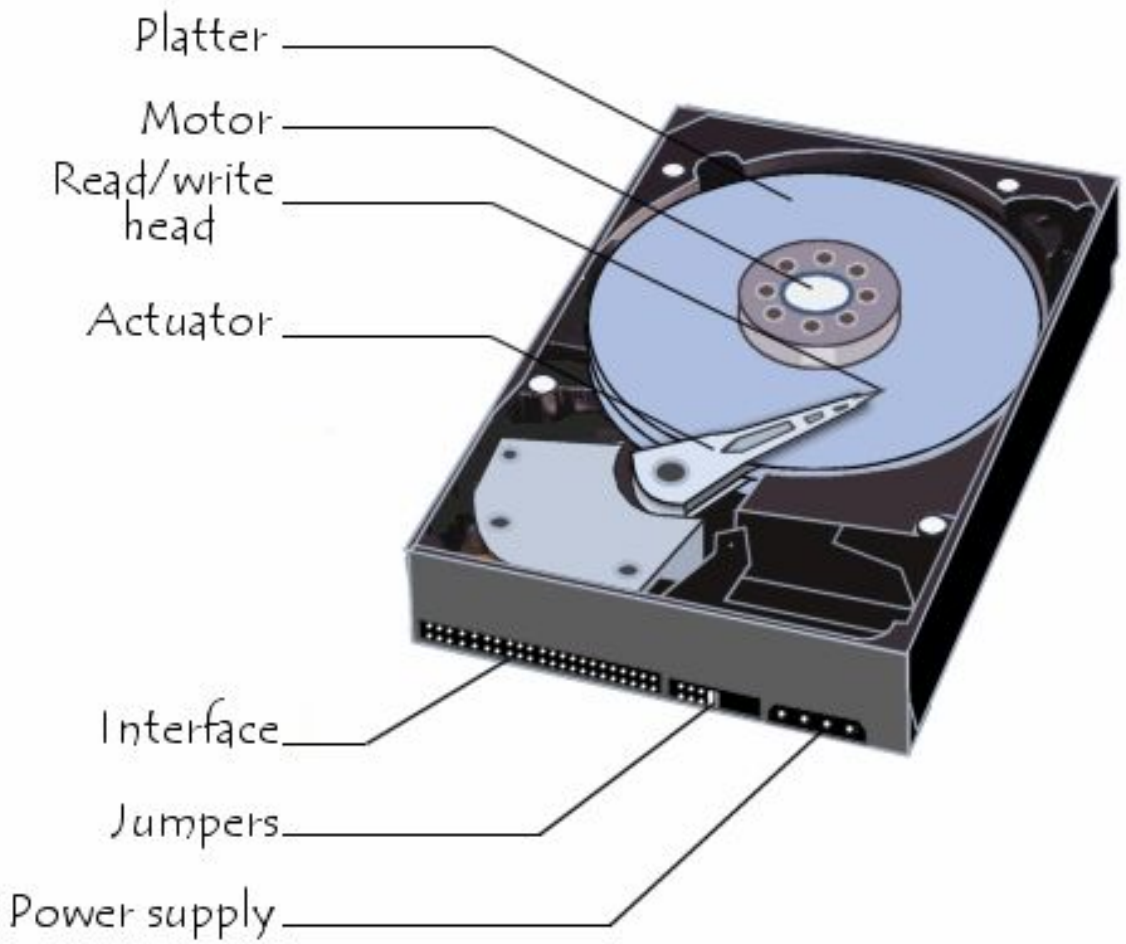
STORAGE DEVICE MEDIA

Magnetic tape
Magnetic floppy disks (obsolete)
Magnetic spinning disks (hard disk drive aka HDD)
    Various capacities & speeds
NAND Flash memory (solid data drive aka SSD)
    Various capacities & speeds
Optical CDs, DVDs

Future research: ceramic, quartz glass, DNA

STORAGE DEVICE: INTERNALS

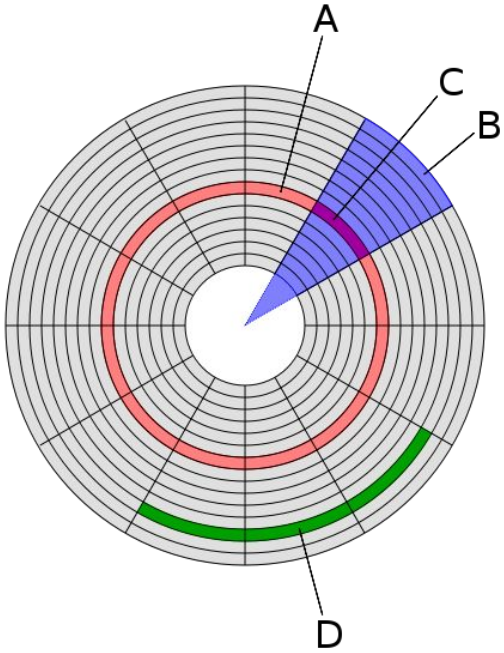



Platter
Motor
Read/write head
Actuator
Interface
Jumpers
Power supply

HARD DISK DRIVE (HDD) INTERNALS

This is one side of a single platter
A: track (<0.5M per radius inch)
B: sector (<100 per track)
C: track sector
D: a file (logical entity)

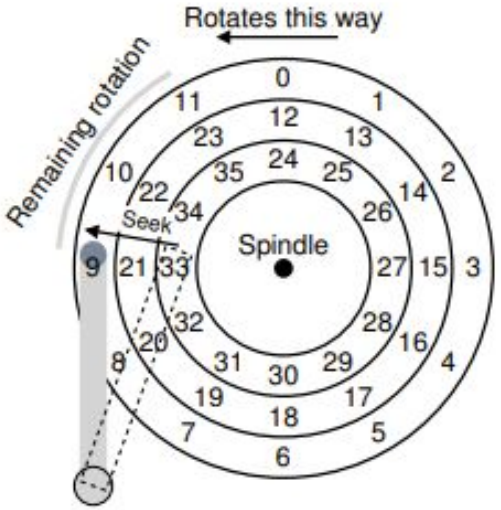
Spindle thru' the center
Rotates at: 7.5k to 15k rpm
R/W head for each side of platter



Seek: move head to the target track

Rotate: wait for target sector to be under head

Transfer: access data



HDD LATENCY: SEEK, ROTATE, TRANSFER

Seek cost for the head:
- Not purely linear cost
- Accelerate, coast, decelerate, & settle
- Settling alone can take 0.5 - 2 ms

Entire seeks often takes 4 - 10 ms
- Average seek = 1/3 of max seek

Depends on rotations per minute (rpm)
- 7.2k rpm is common, 15k rpm is high end

Average rotation – half of a rotation

HDD:
100+ MB/s is typical for max transfer rate

DISK ACCESS TIME

- Disk Parameters
  - Advertised average seek time is 12 ms
  - Disk spins at 7200 RPM
  - Transfer rate is 125 MB/sec
- Assume idle disk (i.e., no queuing delay)

```
Disk Access Time=seek time +
                 rotational delay +
                   transfer time
```

DISK ACCESS TIME

- Disk Parameters
  - Advertised average seek time is 12 ms
  - Disk spins at 7200 RPM
  - Transfer rate is 125 MB/sec

Assume 4K sectors

- Assume idle disk (i.e., no queuing delay)
- Q1: What is the total time to read 500 random sectors?
- Q2: What is the total time to read 500 sequential sectors (assume on same track)?

Discuss with your neighbors for 3 mins…

Q1

- What is the total time to read 500 random sectors?

Q2

- What is the total time to read 500 sequential sectors (assume on same track)?

BANDWIDTH

What is the random access bandwidth?

Sequential bandwidth?

RANDOM VS. SEQUENTIAL

How much difference between random and sequential IO speeds on hard drives?

So, always optimize for sequential access

An example: B-trees vs. log-structured merge trees?

SSD V HDD

	HDD	SSD
Moving parts	Many parts that can fail. Heavier. Damaged by shock/vibration.	None. Faster, quieter, lighter & smaller
Power & cooling	Power to move parts; mechanical parts generate heat	SATA: no cooling needed. NVMe: heat-sink or airflow to avoid thermal throttling
Wear out	magnetic bits don't	Cannot overwrite! Erase + write Finite #writes of flash DWPD (drive writes per day) FTL: wear-leveling
Data loss when powered off	5-10 years; mechanical parts fail before magnetic data loss	1-5 years; power up at least once a year
Cost	Cheaper	Expensive

HDD VS. SSD SPEEDS

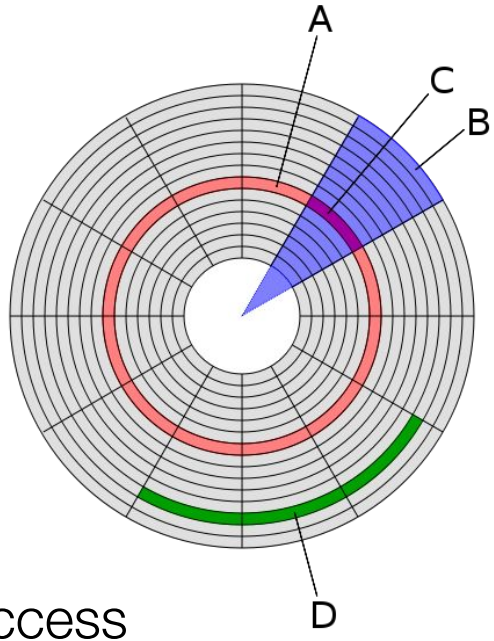
Modern HDD vs NVMe SSD

Metric	Modern 7200-RPM HDD	Modern NVMe SSD
Sequential read	~250–300 MB/s	~7–15 GB/s
Sequential write	~250–300 MB/s	~5–12 GB/s
Random 4 KB read	~150–200 IOPS	~1–2+ million IOPS
Random 4 KB write	~300–400 IOPS	~0.2–2+ million IOPS
Typical access latency	~4–10 ms	~50–100 µs

Sequential bandwidth differs by tens of times; random I/O performance and latency differ by roughly 2–4 orders of magnitude.

DISK SCHEDULING

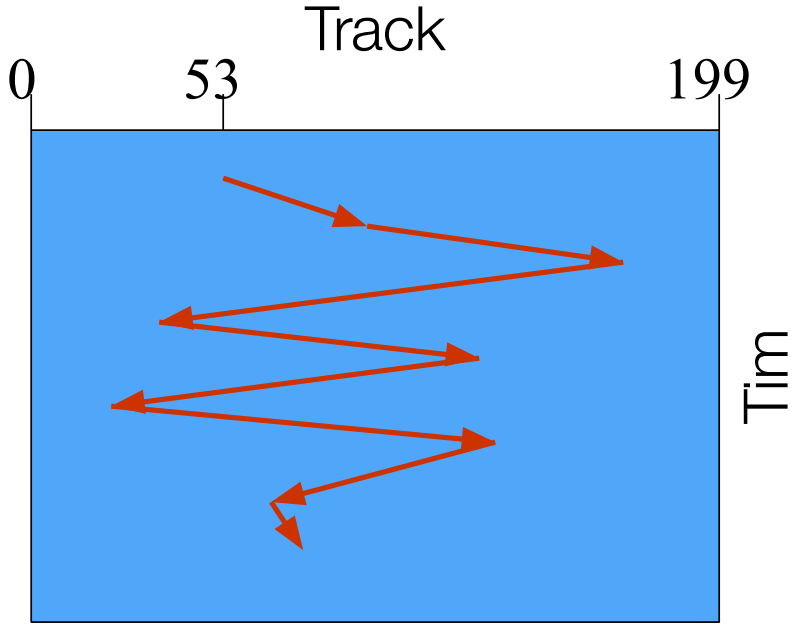
- Which disk request is serviced first?
  - FCFS
  - Shortest seek time first
  - SCAN (Elevator)
  - C-SCAN (Circular SCAN)



Disk Scheduling Decision — Given a series of access requests, on which track should the disk arm be placed next to maximize fairness, throughput, etc?

FCFS

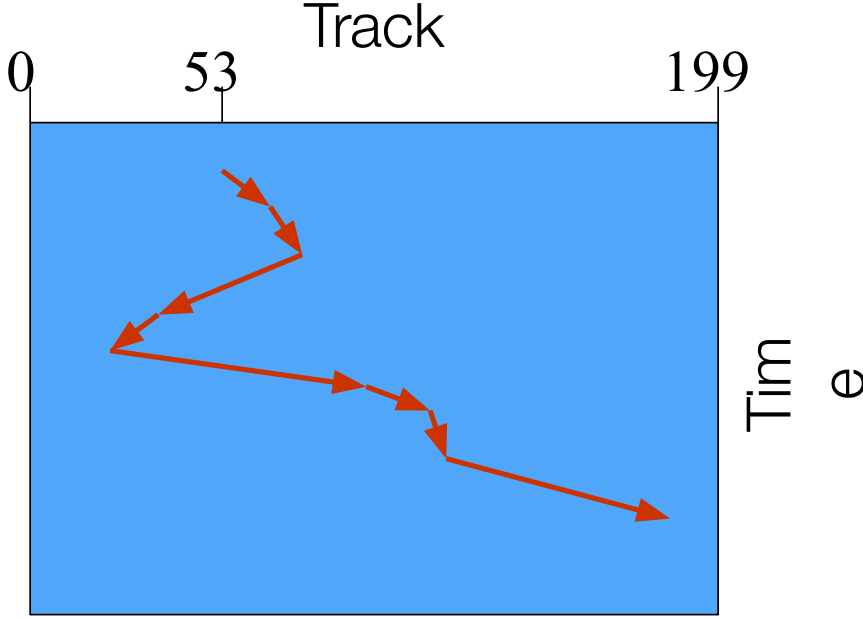
- Method
  - First come first serve
- Pros?
  - Fairness among requests
  - In the order applications expect
- Cons?
  - Arrival may be on random spots on the disk (long seeks)
  - When is it particularly bad?



98, 183, 37, 122, 14, 124, 65, 67

SSTF (SHORTEST SEEK TIME FIRST)

- Method
  - Pick the one closest on disk (greedy approach)
- Pros?
  - Tries to minimize seek time
- Cons?
  - ???

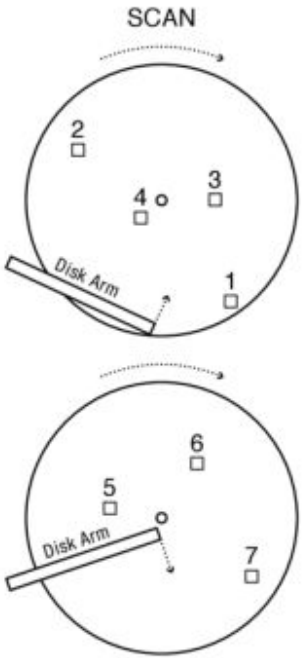


98, 183, 37, 122, 14, 124, 65, 67
(65, 67, 37, 14, 98, 122, 124, 183)

ELEVATOR (SCAN)

- Move outer to inner – service all requests along the way
- Move inner to outer – service all along the way

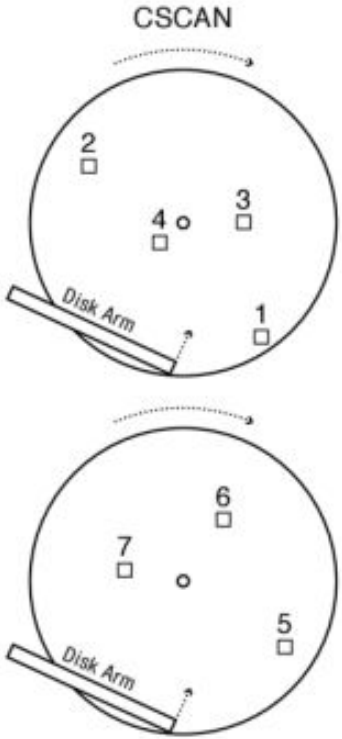
- Adv compared to SSTF:
  - Bounded time for each request



CIRCULAR SCAN (C-SCAN)

Like SCAN
But, wrap around (i.e., only one direction)

- Adv over SCAN
  - By seeking to opposite side, moves head to where pending requests are likely to be denser
  - More fair
- Cons
  - Do nothing on the return (i.e., higher overhead)

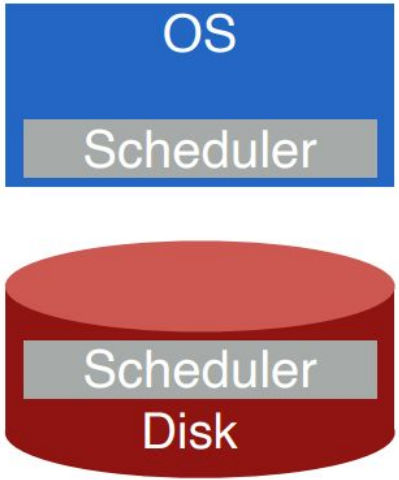


Who does the Scheduling?

The OS?

The disk itself?

Both?



Linux I/O Schedulers

- What disk (I/O) schedulers are available in Linux?

```
$ cat /sys/block/sda/queue/scheduler
noop deadline [cfq]
```

- As of Linux 2.6.10, it is possible to change the IO scheduler for a given block device on the fly!
- How to enable a specific scheduler?

```
$ echo SCHEDNAME > /sys/block/DEV/queue/scheduler
```

  - SCHEDNAME = Desired I/O scheduler
  - DEV = device name (e.g., sda)

Linux NOOP Scheduler

- Insert all incoming I/O requests into a simple FIFO
- Merges duplicate requests
- When would this be useful?

Linux NOOP Scheduler

- Insert all incoming I/O requests into a simple FIFO
- Merges duplicate requests (results can be cached)
- When would this be useful?
  - Solid State Drives! Avoids scheduling overhead
  - Scheduling is handled at a lower layer of the I/O stack (e.g., Disk firmware, RAID Controller, Network-Attached)
  - Host doesn't actually know details of sector positions

Linux Deadline Scheduler

- Imposes a deadline on all I/O operations to prevent starvation of requests
- Maintains 4 queues:
  - 2 Sorted Queues (R, W), order by Sector
  - 2 Deadline Queues (R, W), order by Exp Time
- Scheduling Decision:
  - Check if 1st request in deadline queue has expired.
  - Otherwise, serve request(s) from Sorted Queue.
  - Prioritizes reads (DL=500ms) over writes (DL=5s) .Why?

LINUX CFQ SCHEDULER

- CFQ = Completely Fair Queueing!
- Maintain per-process queues.
- Allocate time slices for each queue to access the disk
- I/O Priority dictates time slice, # requests per queue
- Asynchronous requests handled separately — batched together in priority queues

WHAT HAPPENS?

Assume 2 processes each calling read() with C-SCAN

```
void reader(int fd) {
char buf[1024]; int rv;
while((rv = read(fd, buf)) != 0)  {
assert(rv);
// takes short time, e.g., 1ms
process(buf, rv);
}}
```

P1: read 100, 101

P2: read 900, 901

After 1 ms

P1: read 102, 103

P2: read 902, 903

WORK CONSERVATION

Work conserving schedulers always try to do work if there's work to be done

Sometimes, it's **better to wait** instead if system anticipates another request will arrive

Possible improvements from I/O merging

LINUX ANTICIPATORY SCHEDULER

- Deceptive Idleness: A process appears to be finished reading from disk, but is actually processing data. Another (nearby) request is coming soon!
- Bad for synchronous read workloads because seek time is increased.
- Anticipatory Scheduling: Idle for a few milliseconds after a read operation in *anticipation* of another close-by read request.

SUMMARY

Disks: specific geometry with platters, spindle, tracks, sector, head, etc

DAT = seek time + rotation delay + transfer time

Sequential bandwidth is much higher than random bandwidth

Scheduling approaches: FCFS, SSTF, SCAN, C-SCAN

Schedulers are at multiple layers of the stack

Next: upper layers — the file system!